PowerLASCOPF.jl Development Guide

Understanding the Sienna Ecosystem Architecture

Package Hierarchy

```
InfrastructureSystems.jl (Base utilities & abstract types)
            ↓
PowerSystems.jl (Data structures & power system modeling)
            ↓
PowerSimulations.jl (Optimization & simulation framework)
            ↓
PowerLASCOPF.jl (Your LASCOPF extension)
```

Key Design Patterns in Sienna

1. Abstract Type Hierarchy

The Sienna ecosystem uses extensive abstract type hierarchies:

```julia
# InfrastructureSystems.jl defines base abstractions
abstract type InfrastructureSystemsType end
abstract type InfrastructureSystemsContainer end
abstract type Component <: InfrastructureSystemsType end

# PowerSystems.jl extends these
abstract type Device <: Component end
abstract type Generator <: Device end
abstract type ThermalGen <: Generator end
```

2. Struct Extensions Pattern

Each package extends structs from lower levels:

```julia
# PowerSystems.jl defines components
struct ThermalStandard <: ThermalGen
    # Core power system data
end

# PowerSimulations.jl defines formulations
abstract type AbstractThermalFormulation <: AbstractDeviceFormulation end
struct ThermalBasicUnitCommitment <: AbstractThermalFormulation end
```

3. Multiple Dispatch for Specialization

Functions are specialized using multiple dispatch:

```julia
# Generic function
construct_device!(container, sys, model, device, formulation)

# Specialized methods
construct_device!(container, sys, model, device::ThermalGen, ::ThermalBasicUnitCommitm
construct_device!(container, sys, model, device::RenewableGen, ::RenewableFullDispatch
```

Recommended PowerLASCOPF.jl Structure

1. Package Structure

```
PowerLASCOPF.jl/
├── src/
│   ├── PowerLASCOPF.jl              # Main module file
│   ├── core/
│   │   ├── lascopf_formulations.jl   # Your LASCOPF formulation types
│   │   ├── lascopf_containers.jl     # Extended optimization containers
│   │   └── lascopf_models.jl         # Model extensions
│   ├── devices/
│   │   ├── generators.jl             # Generator extensions for LASCOPF
│   │   ├── branches.jl               # Branch/line extensions
│   │   └── loads.jl                  # Load extensions
│   ├── formulations/
│   │   ├── device_formulations.jl    # Device-specific formulations
│   │   ├── network_formulations.jl   # Network formulations
│   │   └── objective_functions.jl    # LASCOPF-specific objectives
│   ├── models/
│   │   ├── decision_models.jl        # Extended decision models
│   │   └── simulation_models.jl      # Extended simulation models
│   └── utils/
│       ├── contingency_analysis.jl   # Contingency handling
│       └── security_constraints.jl   # Security constraint utilities
├── test/
├── docs/
└── Project.toml
```

2. Main Module Pattern

```julia
# src/PowerLASCOPF.jl
module PowerLASCOPF

# Import required packages
using PowerSystems
using PowerSimulations
using InfrastructureSystems
using JuMP
using HiGHS  # or your preferred solver

# Import types for extension
import PowerSimulations:
    AbstractDeviceFormulation,
    AbstractBranchFormulation,
    DecisionModel,
    OptimizationContainer

# Import functions for overloading
import PowerSimulations:
    construct_device!,
    construct_network!,
    objective_function!,
    add_constraints!,
    add_variables!

# Export your new types and functions
export LASCOPFFormulation,
       SecurityConstrainedFormulation,
       ContingencyAnalysisModel

# Include all source files
include("core/lascopf_formulations.jl")
include("core/lascopf_containers.jl")
include("devices/generators.jl")
include("formulations/device_formulations.jl")
# ... other includes

end # module
```

3. Formulation Extension Pattern

```julia
# src/core/lascopf_formulations.jl

# Define your LASCOPF-specific formulation types
abstract type AbstractLASCOPFFormulation <: AbstractDeviceFormulation end

struct LASCOPFThermalGeneration <: AbstractLASCOPFFormulation end
struct LASCOPFRenewableGeneration <: AbstractLASCOPFFormulation end

# Define network formulations for security constraints
abstract type AbstractSecurityConstrainedFormulation <: AbstractBranchFormulation end

struct SecurityConstrainedACPF <: AbstractSecurityConstrainedFormulation end
struct SecurityConstrainedDCPF <: AbstractSecurityConstrainedFormulation end
```

4. Method Overloading Pattern

```julia
# src/formulations/device_formulations.jl

# Extend existing functions with your formulations
function PowerSimulations.construct_device!(
    container::OptimizationContainer,
    sys::PSY.System,
    model::DeviceModel{T, LASCOPFThermalGeneration},
    network_model::NetworkModel{<:PM.AbstractPowerModel},
) where {T <: PSY.ThermalGen}

    # Call parent method first
    PowerSimulations.construct_device!(
        container,
        sys,
        DeviceModel(T, ThermalBasicUnitCommitment()),
        network_model
    )

    # Add your LASCOPF-specific constraints
    add_security_constraints!(container, sys, model)
    add_contingency_constraints!(container, sys, model)

    return
end

function add_security_constraints!(
    container::OptimizationContainer,
    sys::PSY.System,
    model::DeviceModel{T, LASCOPFThermalGeneration}
) where {T <: PSY.ThermalGen}
    # Your security constraint implementation
    devices = PSY.get_components(T, sys)

    # Add N-1 contingency constraints
    for device in devices
        # Implementation specific to LASCOPF
    end
end
```

5. Container Extension Pattern

```julia
# src/core/lascopf_containers.jl

# Extend optimization container for LASCOPF-specific variables
mutable struct LASCOPFContainer <: PowerSimulations.AbstractOptimizationContainer
    base_container::OptimizationContainer
    contingency_variables::Dict{String, JuMP.VariableRef}
    security_constraints::Dict{String, JuMP.ConstraintRef}

    function LASCOPFContainer(base::OptimizationContainer)
        return new(base, Dict{String, JuMP.VariableRef}(), Dict{String, JuMP.Constrain
    end
end

# Delegate methods to base container
PowerSimulations.get_optimization_container(c::LASCOPFContainer) = c.base_container
```

Integration Best Practices

1. Follow Sienna Conventions

- Use the same naming patterns (`construct_`, `add_`, `get_`, `set_`)
- Follow the same abstract type hierarchy patterns
- Use similar documentation and testing patterns

2. Leverage Existing Infrastructure

```julia
# Don't reimplement - extend existing functionality
function create_lascopf_model(sys::PSY.System)
    # Use PowerSimulations' existing model creation
    template = ProblemTemplate()

    # Add your LASCOPF formulations
    set_device_model!(template, ThermalStandard, LASCOPFThermalGeneration())
    set_device_model!(template, RenewableDispatch, LASCOPFRenewableGeneration())
    set_network_model!(template, SecurityConstrainedDCPF)

    return DecisionModel(template, sys)
end
```

3. Maintain Compatibility

- Always call parent methods when extending
- Use `import` for functions you're overloading
- Test compatibility with different PowerSystems versions

4. Documentation Integration

- Follow Sienna's documentation style
- Include examples that work with existing PowerSystems data
- Document which base methods you're extending

Next Steps

1. **Show me your partial code** - I can provide specific guidance on structure and implementation
2. **Identify key extension points** - What specific LASCOPF functionality are you implementing?
3. **Plan testing strategy** - Using PowerSystems test cases and your own LASCOPF-specific tests

The key is to leverage the existing Sienna architecture rather than fighting it - your PowerLASCOPF.jl should feel like a natural extension of the ecosystem!